

ECOM08A

Inteligência Artificial

Prof. João Paulo R. R. Leite
joaopaulo@unifei.edu.br

Em **problemas de busca**, como encontrar o caminho mais curto em um grafo ou resolver quebra-cabeças complexos, os algoritmos de busca desempenham um papel crucial.

Métodos de **busca cega**, como a Busca em Largura (BFS) e a Busca em Profundidade (DFS), vistos na última aula, exploram o espaço de busca de maneira sistemática, mas **sem qualquer informação adicional** sobre a localização da solução. Isso pode torná-los menos eficientes em termos de tempo e espaço, especialmente em grandes espaços de busca.

Por outro lado, métodos de **busca informada ou heurística** utilizam **informações adicionais** – que chamamos de heurísticas - para **guiar a busca** de maneira mais inteligente e eficiente.

Isso pode reduzir significativamente o número de estados explorados e acelerar a busca pela solução.

Heurísticas: Conceito e Importância

A palavra "**heurística**" vem do grego "εὕρισκω" (pronuncia-se "*evrisko*"), que significa "**encontrar**" ou "**descobrir**".

No contexto da ciência da computação e inteligência artificial, heurísticas são métodos que ajudam a encontrar soluções de forma eficiente, especialmente em problemas complexos **onde a busca exaustiva seria impraticável**.



Heurística, portanto, é uma **estimativa** que fornece uma "boa ideia" de quão longe estamos da solução ou quão caro pode ser alcançá-la, ajudando a guiar a busca em direção à solução mais rapidamente. Ela é uma **função** que estima o **custo de se mover de um estado atual em um espaço de busca para o estado objetivo**.

Em termos práticos, uma heurística é usada para **avaliar quais estados são mais promissores** e, portanto, devem ser **explorados primeiro**. É uma função que, a cada etapa da busca, **classifica alternativas** com base nas informações disponíveis para decidir qual ramificação seguir no próximo passo.

As heurísticas são o "**coração**" dos métodos de busca informada. Elas **fornece**m a **direção** para os algoritmos de busca, permitindo que eles **encontrem soluções de maneira mais eficiente**.

A qualidade de uma heurística pode determinar o sucesso ou fracasso na resolução de problemas complexos em tempo viável. Portanto, o desenvolvimento e a aplicação de boas heurísticas são fundamentais na Inteligência Artificial e em várias áreas de otimização e planejamento.

Por exemplo, em roteamento de mapas, uma heurística simples como a **distância Euclidiana** pode ser rápida, mas pode não considerar adequadamente as características do terreno. Uma heurística mais sofisticada que considere estradas e tráfego pode ser mais lenta para calcular, mas mais eficaz em encontrar a rota mais rápida.

Para que uma heurística seja eficaz, ela deve ter certas propriedades:

- **Admissibilidade:** Uma heurística é considerada admissível se ela nunca superestima o custo de alcançar o estado objetivo.
 - Por exemplo, imagine um problema onde você deseja encontrar o caminho mais curto em uma grade 2D. Se a heurística for a "distância em linha reta" (ou **distância Euclidiana**) até o objetivo, ela será admissível porque a distância em linha reta nunca será maior do que o caminho real (que pode envolver curvas).
- **Consistente (ou Monotônica):** A heurística deve satisfazer a condição de consistência, onde o custo estimado para alcançar o objetivo ($h(n)$) não deve ser maior do que o custo de ir a um estado vizinho $c(n, n')$ mais o custo estimado desse estado até o objetivo ($h(n')$). Ou seja: $h(n) \leq c(n, n') + h(n')$
 - Consistência garante que, à medida que o algoritmo expande nós, ele não precise reavaliar nós já visitados, mantendo a eficiência da busca.

- Quando a heurística é **admissível** e **consistente**:
 - Algoritmos como a **busca A*** garantem que a solução encontrada será **ótima**. Isso significa que, se a heurística não superestima o custo real e mantém a consistência, o algoritmo sempre encontrará o caminho mais curto ou a **melhor solução possível**.
- Heurísticas em **Algoritmos Aproximados** ou **Não Ótimos**:
 - Quando a heurística não precisa ser admissível ou consistente, como na **busca gulosa**, eles utilizam heurísticas para priorizar caminhos que parecem promissores, mas não garantem que a solução encontrada seja a ótima.
 - A principal vantagem aqui é a velocidade: a **busca é mais rápida** porque não se perde tempo explorando caminhos que a heurística sugere serem menos promissores.
 - Nesse cenário, as heurísticas podem levar a uma solução que **não é a melhor possível**, mas que ainda é **suficientemente boa** e encontrada em um tempo razoável. Isso é particularmente útil em problemas onde a busca exaustiva seria impraticável ou onde uma solução ótima não é absolutamente necessária.

Il meglio è nemico del bene

A expressão acima, que significa “o ótimo é inimigo do bom” pode ser verdadeira em muitos casos, também na **computação**.

Em problemas onde o espaço de busca é vasto ou a complexidade é alta, buscar a solução ótima pode exigir um tempo e recursos computacionais impraticáveis. Nesses casos, **buscar a solução ótima pode ser inviável**.

Além disso, Em muitas situações práticas, especialmente aquelas que envolvem **decisões em tempo real**, é mais importante encontrar uma solução viável rapidamente do que encontrar a solução perfeita, mas atrasada.

Busca A^* (A-estrela)

É um algoritmo de **busca informada** utilizado para encontrar o caminho mais curto entre dois pontos em um grafo, levando em consideração tanto o **custo real do caminho percorrido** quanto uma **estimativa heurística do custo restante** para alcançar o objetivo. Nele temos:

$g(n)$: O custo real acumulado do início até o nó n .

$h(n)$: A heurística, que estima o custo restante do nó n até o objetivo.

$f(n) = g(n) + h(n)$: A função de avaliação do A^* , que combina o custo real até n com a estimativa do custo restante.

O nó com o **menor valor** de **$f(n)$** é escolhido como **próximo a ser explorado**.

Passo a passo:

1. **Inicialização:** Começa com o nó inicial, atribuindo-lhe um custo inicial $g(n) = 0$ e calculando $f(n) = h(n)$.
2. **Exploração:** Em cada passo, expande-se o nó com o menor valor de $f(n)$.
3. **Expansão:** Para cada nó vizinho, calcula o custo $g(n)$ e atualiza $f(n)$. Se um caminho melhor para um nó vizinho for encontrado, esse nó é atualizado.
4. **Objetivo:** O processo continua até que o nó objetivo seja expandido, momento em que o caminho mais curto é encontrado.

Vamos considerar um exemplo onde queremos encontrar o caminho mais curto em uma **grade 2D** de um ponto inicial até um ponto objetivo, evitando obstáculos. A grade 2D será representada por uma matriz onde 0 representa um espaço livre e 1 representa um obstáculo. São permitidos movimentos horizontal ou vertical, com custo de 1 unidade por movimento.

```

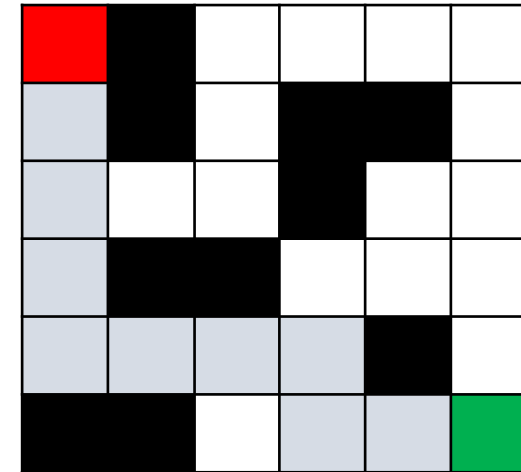
# Exemplo de grade 2D
grid = [
    [0, 1, 0, 0, 0, 0],
    [0, 1, 0, 1, 1, 0],
    [0, 0, 0, 1, 0, 0],
    [0, 1, 1, 0, 0, 0],
    [0, 0, 0, 0, 1, 0],
    [1, 1, 0, 0, 0, 0]
]

start = (0, 0) # Inicio na posicao (0, 0)
goal = (5, 5) # Objetivo na posicao (5, 5)

# Executa o A*
path = a_star_search(grid, start, goal)

# Mostra o caminho encontrado
if path:
    print("Caminho encontrado:")
    for step in path:
        print(step)
else:
    print("Nenhum caminho encontrado.")

```



O mapa acima representa graficamente o exemplo do código ao lado. A célula vermelha (0,0) é o início, e a célula verde (5,5) é o final. A resposta dada pelo algoritmo está representada em **azul claro** na imagem.

```

Caminho encontrado:
(0, 0)
(1, 0)
(2, 0)
(3, 0)
(4, 0)
(4, 1)
(4, 2)
(4, 3)
(5, 3)
(5, 4)
(5, 5)

```

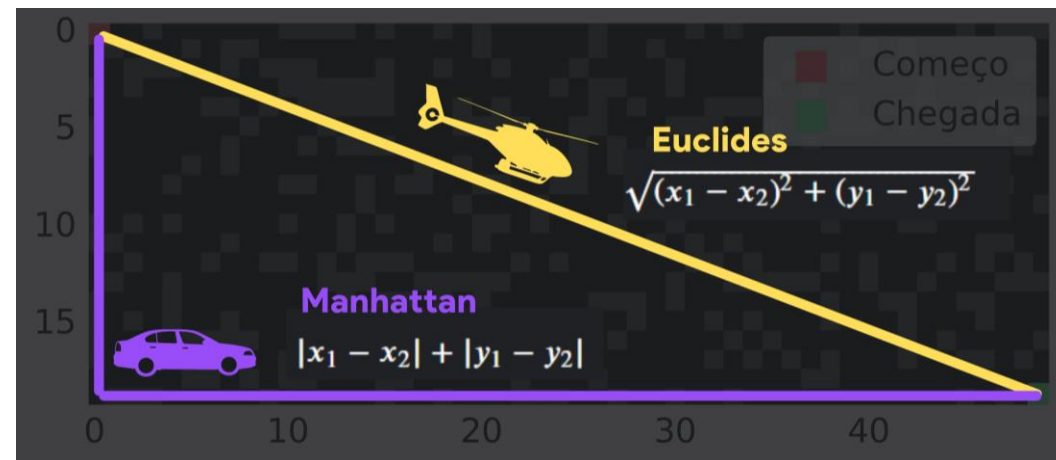
Com o cenário pronto, precisamos decidir... qual será o nosso Valor Heurístico?

Para isso temos duas opções simples: a distância de Euclides, ou a distância Manhattan. Iremos utilizar a Distância Manhattan.

Mas, qual a razão da escolha?

Imagine uma rota de GPS para dois veículos, uma para um carro e outra para um helicóptero. A Distância Euclidiana seria uma reta para uma possível rota de helicóptero (na qual não haveria preocupação com as ruas já que é um veículo voa) e a **Distância Manhattan** seria um segmento de retas na vertical quanto na horizontal semelhante a uma rota de carro (já que esse obedece o sentido das ruas).

```
# Heurística: Distancia Manhattan
def heuristic(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
```



Também é conhecida como **métrica do táxi**.

```

# Funcao para reconstruir o caminho a partir dos pais
def reconstruct_path(came_from, start, goal):
    current = goal
    path = [current]
    while current != start:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path

# Algoritmo A*
def a_star_search(grid, start, goal):
    rows, cols = len(grid), len(grid[0]) # limites do grid
    queue = PriorityQueue()
    queue.put((0, start)) # inicio com g(n) = 0
    came_from = {} # de onde veio
    cost_so_far = {} # custo atualizado ate agora
    came_from[start] = None
    cost_so_far[start] = 0

    while not queue.empty():
        _, current = queue.get() # pega apenas o no com menor f(n). despreza valor.

        if current == goal: # se for objetivo, termina
            break

        for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]: #para cada movimento possivel
            neighbor = (current[0] + dx, current[1] + dy)
            if 0 <= neighbor[0] < rows and 0 <= neighbor[1] < cols: #dentro do grid
                if grid[neighbor[0]][neighbor[1]] == 1: # se for obstaculo
                    continue

            new_cost = cost_so_far[current] + 1 # novo custo = custo atual + 1

            # se vizinho ainda nao tem custo ou custo encontrado for menor que anterior
            if neighbor not in cost_so_far or new_cost < cost_so_far[neighbor]:
                cost_so_far[neighbor] = new_cost # g(n)
                priority = new_cost + heuristic(neighbor, goal) # f(n) = g(n) + h(n)
                queue.put((priority, neighbor)) # coloca na fila de prioridades
                came_from[neighbor] = current # atualiza no anterior de "neighbor"

    return reconstruct_path(came_from, start, goal)

```

A função **reconstruct_path** reconstrói o caminho a partir dos nós pais, começando no objetivo e retrocedendo até o início. Depois ela o reverte, para ficar na ordem correta e retorna.

A função **a_star_search** implementa a **busca A***. Nela, nós utilizamos uma **fila de prioridade** (PriorityQueue) para sempre expandir o nó com o menor valor de $f(n)$ (custo + heurística). Esse nó estará sempre em seu topo (*heap*).

Ao expandir o nó, o custo acumulado $g(n)$ é calculado para cada nó vizinho válido, e o valor de $f(n)$ é atualizado. Se um caminho melhor é encontrado para um vizinho, o nó é atualizado e inserido novamente na fila de prioridade.

O algoritmo executa até que o nó objetivo (goal) seja visitado (ou terminem os nós da fila).

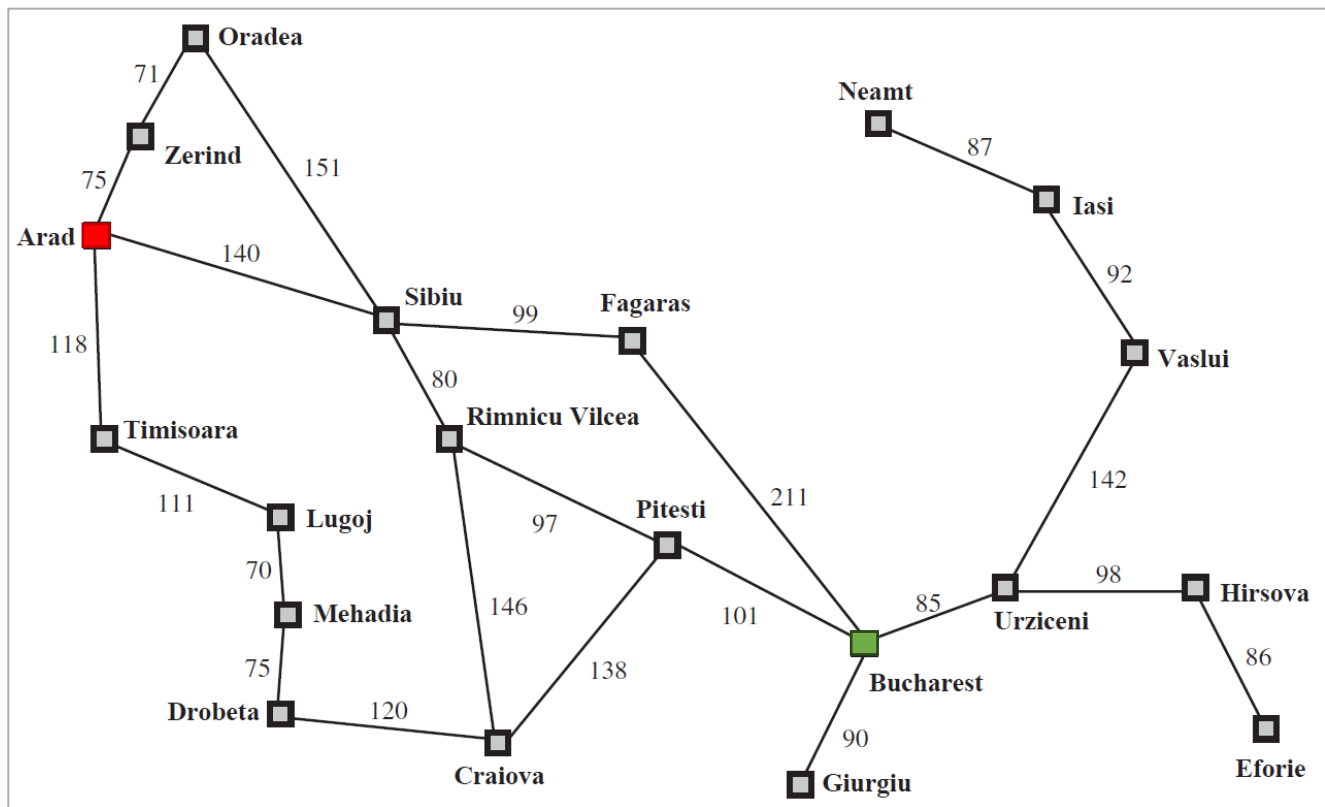
Mas parece o Dijkstra!

Um pouco, mas vejamos as diferenças:

Dijkstra encontra o caminho mais curto de um único ponto de **origem para todos os outros** nós em um grafo. Também pode ser usado para encontrar o caminho mais curto de um ponto de origem a um destino específico. Ele sempre expande o **nó mais próximo do ponto de origem** (aquele com o menor custo acumulado) e atualiza os custos para alcançar os nós vizinhos. **Não usa heurística**. Garante a solução ótima. Completo e determinístico.

A* encontra o caminho mais curto de um ponto de origem até um **destino específico**, usando uma **heurística** para guiar a busca de forma mais eficiente. Ele expande o **nó com o menor valor da função $f(n) = g(n) + h(n)$** , onde $g(n)$ é o custo do caminho desde o ponto de origem até o nó n e $h(n)$ é a **heurística** que estima o custo do nó n até o objetivo. **Usa heurística para guiar a busca**. Garante a solução ótima (se a heurística for admissível e consistente).

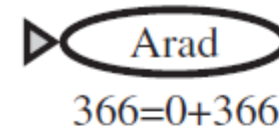
Geralmente, o A* é mais eficiente em encontrar o caminho mais curto em grafos grandes quando se conhece um objetivo específico, pois a heurística guia a busca em direção ao objetivo, **evitando a exploração desnecessária** de nós que estão longe do caminho ótimo.



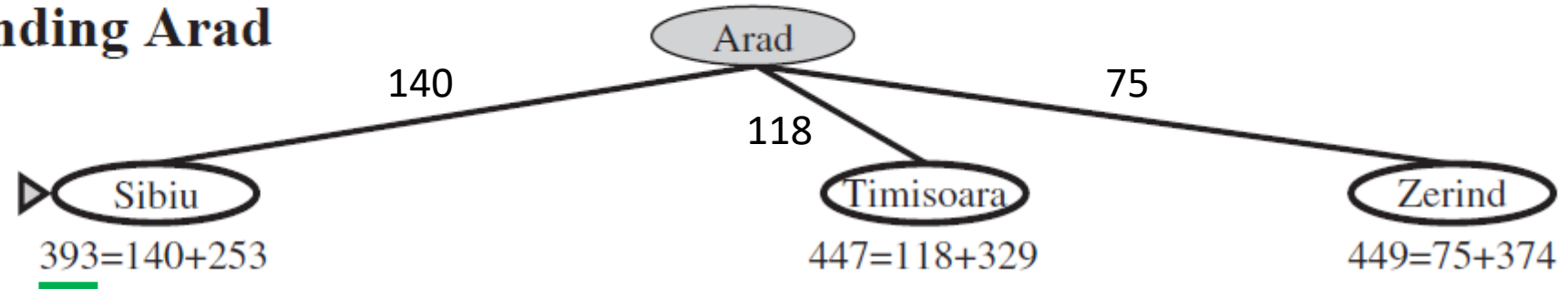
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Tomemos agora um outro exemplo, retirado do livro de [Russel/Norvig](#), em que queremos utilizar a **busca A*** para **viajar pelo caminho mais curto entre Arad e Bucareste**. Este é um mapa simplificado de rodovias da [Romênia](#). Nesse caso, iremos utilizar a Distância Euclidiana como heurística ($h(n)$), sendo que as distâncias em linha reta entre cada cidade e Bucareste estão na tabela à direita. Essa heurística é **admissível**, pois a distância em linha reta nunca superestima a distância real de viagem (sempre será menor ou igual).

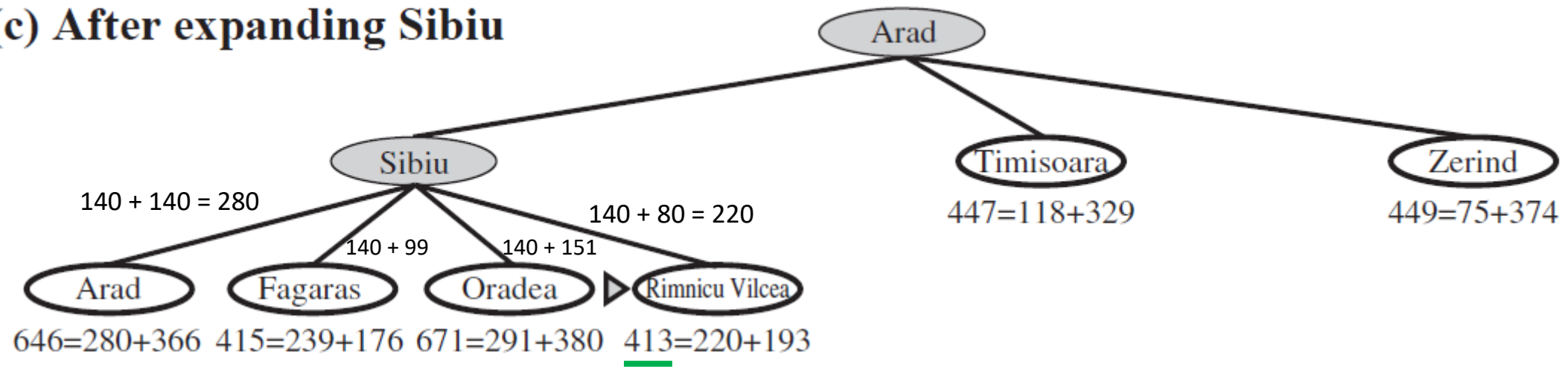
(a) The initial state



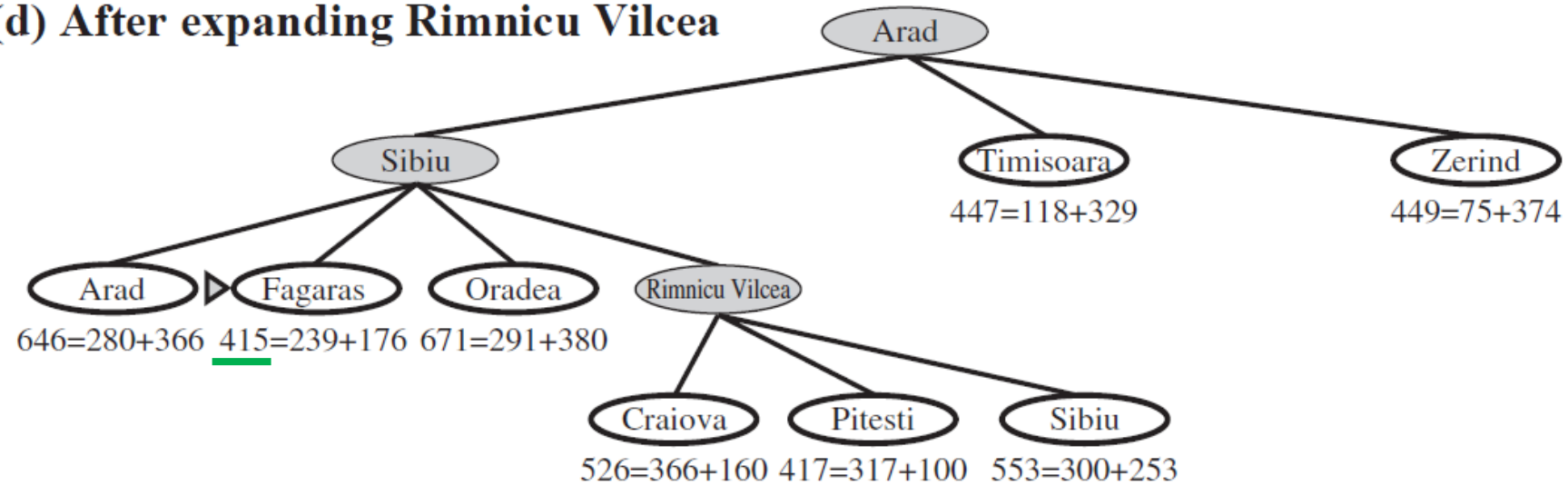
(b) After expanding Arad



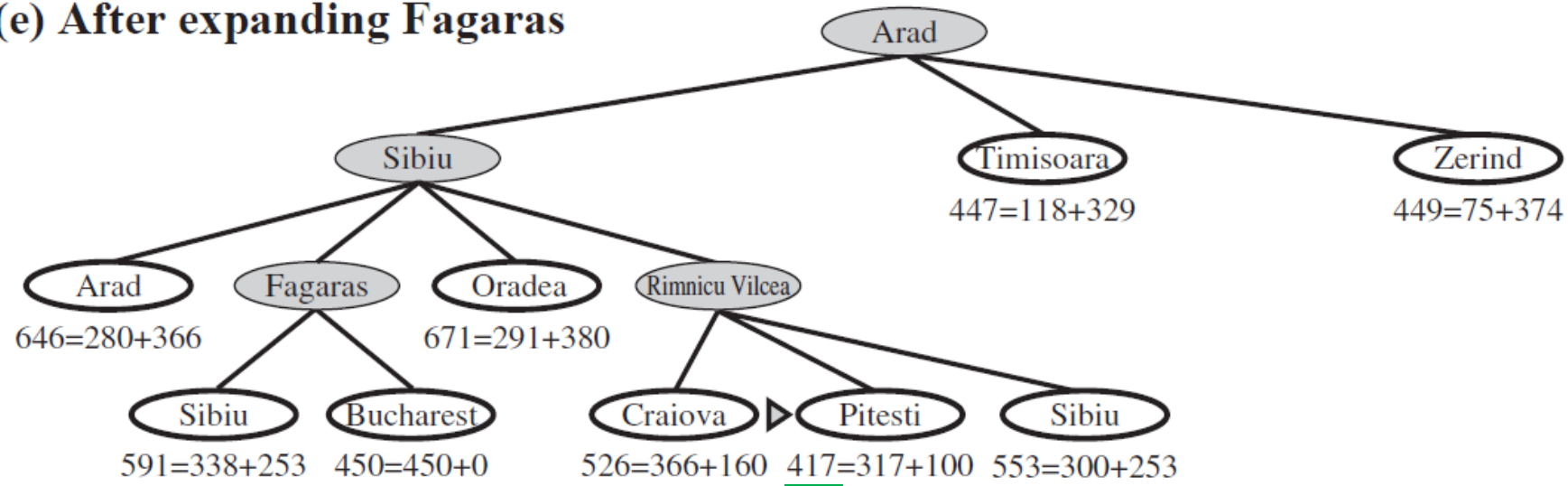
(c) After expanding Sibiu



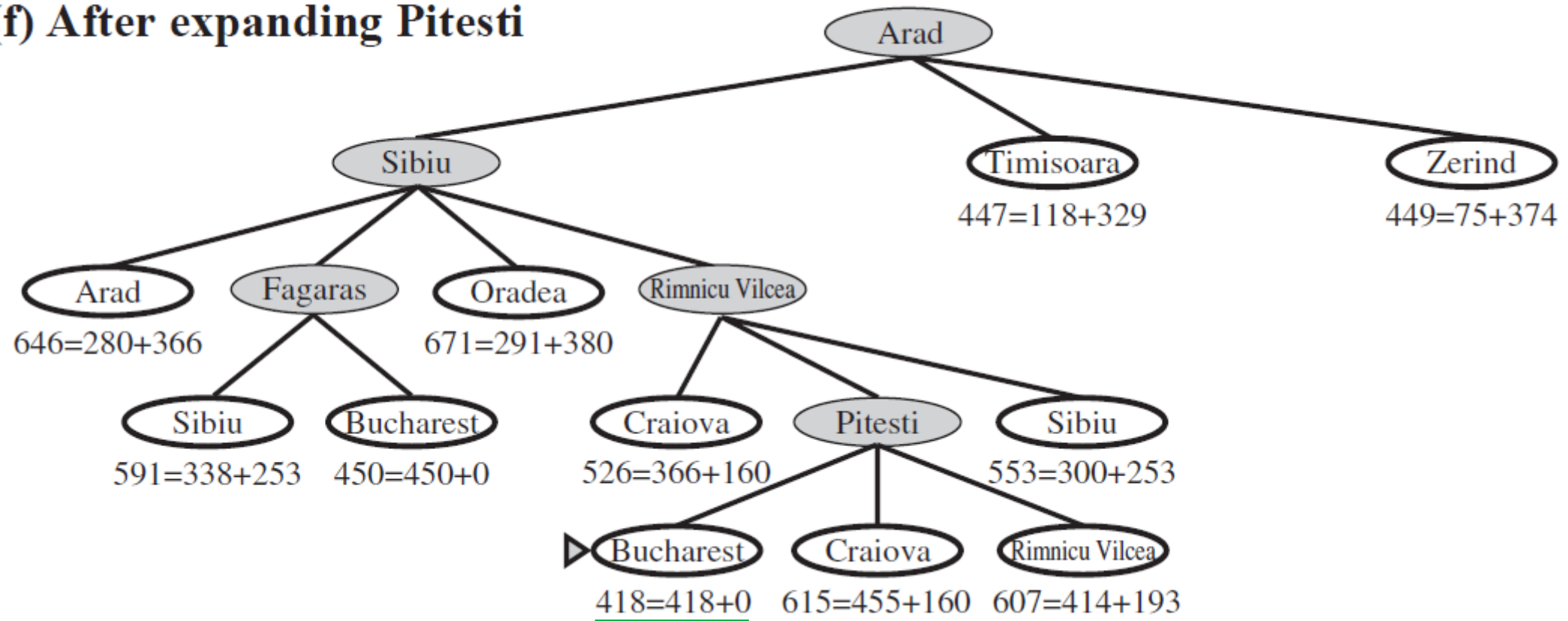
(d) After expanding Rimnicu Vilcea

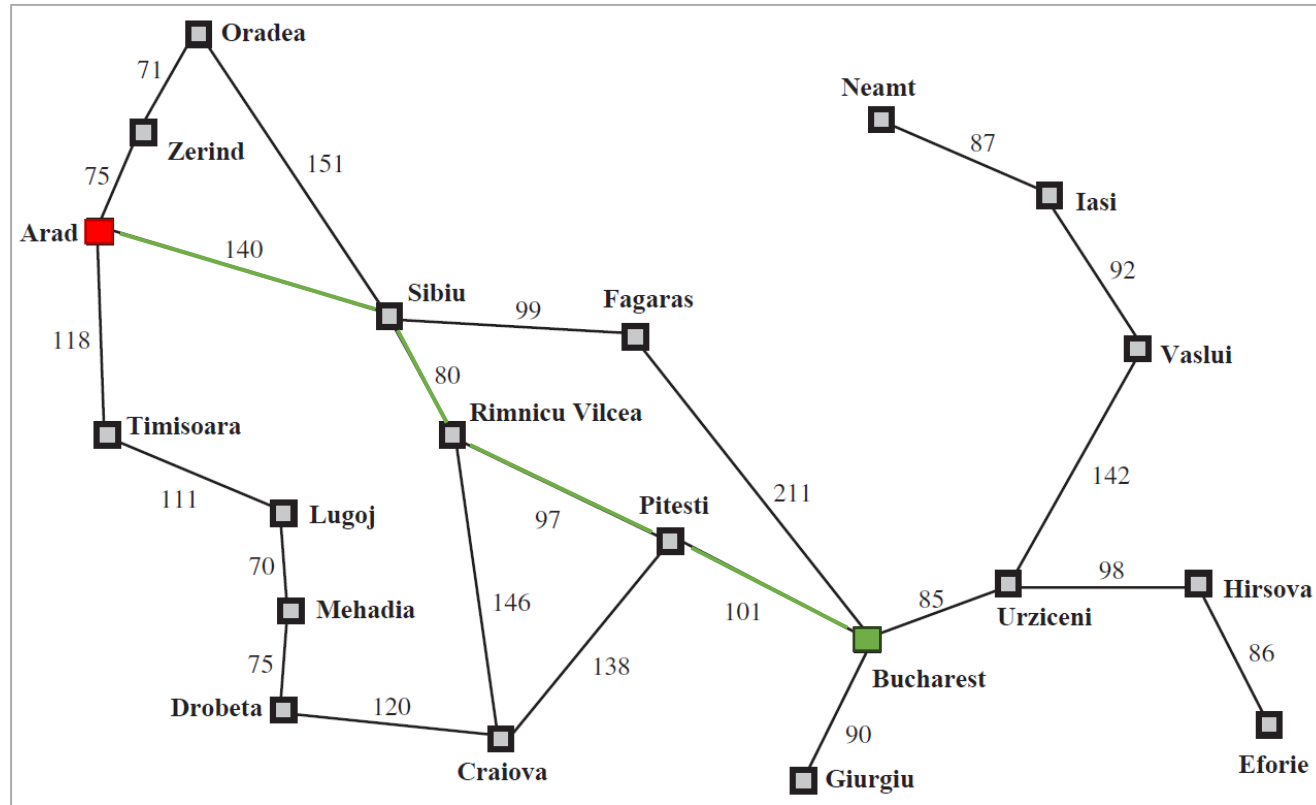


(e) After expanding Fagaras



(f) After expanding Pitesti





O resultado da exploração está na figura acima. A heurística ajuda a guiar a busca, evitando a expansão de nós que não contribuem para o caminho mais curto. Isso torna o A* mais eficiente do que o Dijkstra nesse contexto, pois ele considera tanto o custo real quanto a estimativa do custo restante.

```

from queue import PriorityQueue

# busca A*
def a_star(graph, start, goal, heuristic):
    queue = PriorityQueue()
    queue.put((0, start))
    came_from = {}
    cost_so_far = {}
    came_from[start] = None
    cost_so_far[start] = 0

    while not queue.empty():
        _, current = queue.get()

        if current == goal:
            break

        for neighbor, cost in graph[current].items():
            new_cost = cost_so_far[current] + cost
            if neighbor not in cost_so_far or new_cost < cost_so_far[neighbor]:
                cost_so_far[neighbor] = new_cost
                priority = new_cost + heuristic[neighbor]
                queue.put((priority, neighbor))
                came_from[neighbor] = current

    return reconstruct_path(came_from, start, goal)

# reconstroi caminho a partir do objetivo ate o inicio.
def reconstruct_path(came_from, start, goal):
    current = goal
    path = [current]
    while current != start:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path

```

```

# Grafo das cidades romenas (lista de adjacencias)
graph = {
    'Arad': {'Zerind': 75, 'Sibiu': 140, 'Timisoara': 118},
    'Zerind': {'Arad': 75, 'Oradea': 71},
    'Oradea': {'Zerind': 71, 'Sibiu': 151},
    'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
    'Timisoara': {'Arad': 118, 'Lugoj': 111},
    'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
    'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
    'Drobeta': {'Mehadia': 75, 'Craiova': 120},
    'Craiova': {'Drobeta': 120, 'Rimnicu Vilcea': 146, 'Pitesti': 138},
    'Rimnicu Vilcea': {'Sibiu': 80, 'Craiova': 146, 'Pitesti': 97},
    'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
    'Pitesti': {'Rimnicu Vilcea': 97, 'Craiova': 138, 'Bucharest': 101},
    'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90, 'Urziceni': 85},
    'Giurgiu': {'Bucharest': 90},
    'Urziceni': {'Bucharest': 85, 'Hirsova': 98, 'Vaslui': 142},
    'Hirsova': {'Urziceni': 98, 'Eforie': 86},
    'Eforie': {'Hirsova': 86},
    'Vaslui': {'Urziceni': 142, 'Iasi': 92},
    'Iasi': {'Vaslui': 92, 'Neamt': 87},
    'Neamt': {'Iasi': 87}
}

# Distancia em linha reta ate Bucareste (heurística)
heuristic = {
    'Arad': 366, 'Zerind': 374, 'Oradea': 380, 'Sibiu': 253, 'Timisoara': 329,
    'Lugoj': 244, 'Mehadia': 241, 'Drobeta': 242, 'Craiova': 160, 'Rimnicu Vilcea': 193,
    'Fagaras': 176, 'Pitesti': 100, 'Bucharest': 0, 'Giurgiu': 77, 'Urziceni': 80,
    'Hirsova': 151, 'Eforie': 161, 'Vaslui': 199, 'Iasi': 226, 'Neamt': 234
}

# Executa o A* para encontrar o caminho de Arad a Bucareste
path = a_star(graph, 'Arad', 'Bucharest', heuristic)

print("Caminho encontrado:", path)

```

O código é praticamente o mesmo do problema da grade 2D com obstáculos. Aqui, a função heurística não é necessária, pois temos **valores heurísticos previamente calculados** e armazenados em uma lista.

Embora o A* seja um algoritmo poderoso e frequentemente usado para encontrar o caminho mais curto em muitos problemas, ele pode ter complexidade exponencial no pior caso. Isso ocorre particularmente em problemas com grandes espaços de busca e heurísticas fracas. Em aplicações práticas, o sucesso do A* depende muito da escolha de uma heurística eficaz que equilibre a precisão com a eficiência.

$O(b^d)$ em que b é o fator de ramificação (média de filhos por nó) e d é a profundidade da solução. Isso é exponencial na profundidade da solução, não necessariamente no tamanho do grafo, como o Dijkstra, que é $O(|V|\log|V|+|E|)$.

Portanto, ao implementar A*, é essencial considerar a natureza do problema e a qualidade da heurística para garantir que o algoritmo funcione de forma eficiente e não caia na armadilha de uma exploração exponencial do espaço de busca.

Busca Gulosa

A Busca Gulosa (*Greedy Best-First Search*) é um algoritmo de **busca informada** que, diferentemente do A^* , usa apenas a heurística para decidir qual nó explorar em seguida, sem considerar o custo real já incorrido para alcançar esse nó.

- A Busca Gulosa é um algoritmo que **sempre seleciona o nó que parece ser o mais promissor** em termos de *proximidade ao objetivo*, de acordo com uma **função heurística**.
- O algoritmo é chamado de "guloso" porque toma **decisões que parecem ótimas no momento, sem considerar o impacto dessas decisões no longo prazo**.

Portanto, na Busca Gulosa, a cada passo, o algoritmo escolhe expandir o nó com o menor valor de heurística $h(n)$, que estima a distância do nó n até o objetivo. Dessa maneira, temos que $f(n) = h(n)$, e a busca escolhe o próximo nó a ser explorado com base apenas na heurística $h(n)$.

Vantagens:

- **Simplicidade:** Fácil de implementar, pois só considera a heurística.
- **Velocidade:** Pode ser muito rápida em espaços de busca onde a heurística é boa e os caminhos são simples.

Desvantagens:

- **Não Ótima:** A busca gulosa não garante encontrar o caminho mais curto. Pode facilmente se prender em mínimos locais, onde o nó mais promissor leva a um caminho sem saída ou subótimo.
- **Incompleta:** Em alguns casos, pode não encontrar uma solução, especialmente em espaços de busca com caminhos complexos ou onde a heurística é enganosa.

Diferente de A^* , que é ótima (se heurística for admissível e consistente) e completa (garantindo que a solução será encontrada se existir). Além disso, A^* leva em consideração o custo prévio $g(n)$.

```

# Exemplo de grade 2D
grid = [
    [0, 1, 0, 0, 0, 0],
    [0, 1, 0, 1, 1, 0],
    [0, 0, 0, 1, 0, 0],
    [0, 1, 1, 0, 0, 0],
    [0, 0, 0, 0, 1, 0],
    [1, 1, 0, 0, 0, 0]
]

start = (0, 0) # Inicio na posicao (0, 0)
goal = (5, 5) # Objetivo na posicao (5, 5)

# Executa a Busca Gulosa para encontrar o caminho
path = greedy_best_first_search(grid, start, goal)

# Mostra o caminho encontrado
if path:
    print("Caminho encontrado:")
    for step in path:
        print(step)
else:
    print("Nenhum caminho encontrado.")

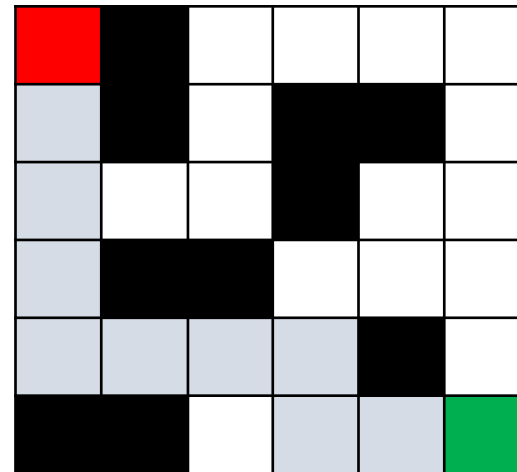
```

Podemos aplicar a busca gulosa ao mesmo problema da navegação entre início e objetivo na grade 2D que resolvemos com a busca A*. Nesse caso, para o exemplo específico criado, a resposta ficou idêntica. Foi mantida a mesma heurística, de Distância Manhattan:

```

# Heurística: Distancia Manhattan
def heuristic(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

```



```

Caminho encontrado:
(0, 0)
(1, 0)
(2, 0)
(3, 0)
(4, 0)
(4, 1)
(4, 2)
(4, 3)
(5, 3)
(5, 4)
(5, 5)

```

```

# Algoritmo de Busca Gulosa
def greedy_best_first_search(grid, start, goal):
    rows, cols = len(grid), len(grid[0])
    queue = PriorityQueue()
    queue.put((0, start))
    came_from = {}
    came_from[start] = None

    while not queue.empty():
        _, current = queue.get()

        if current == goal:
            break

        for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            neighbor = (current[0] + dx, current[1] + dy)
            if 0 <= neighbor[0] < rows and 0 <= neighbor[1] < cols:
                if grid[neighbor[0]][neighbor[1]] == 1: # Obstaculo
                    continue

                if neighbor not in came_from:
                    priority = heuristic(neighbor, goal) # f(n) = h(n)
                    queue.put((priority, neighbor))
                    came_from[neighbor] = current

    return reconstruct_path(came_from, start, goal)

```

```

# Algoritmo A*
def a_star_search(grid, start, goal):
    rows, cols = len(grid), len(grid[0]) # limites do grid
    queue = PriorityQueue()
    queue.put((0, start)) # inicio com g(n) = 0
    came_from = {} # de onde veio
    cost_so_far = {} # custo atualizado ate agora
    came_from[start] = None
    cost_so_far[start] = 0

    while not queue.empty():
        _, current = queue.get() # pega apenas o no com menor f(n). despreza valor.

        if current == goal: # se for objetivo, termina
            break

        for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            neighbor = (current[0] + dx, current[1] + dy)
            if 0 <= neighbor[0] < rows and 0 <= neighbor[1] < cols:
                if grid[neighbor[0]][neighbor[1]] == 1:
                    continue

                new_cost = cost_so_far[current] + 1 # novo custo = custo atual + 1

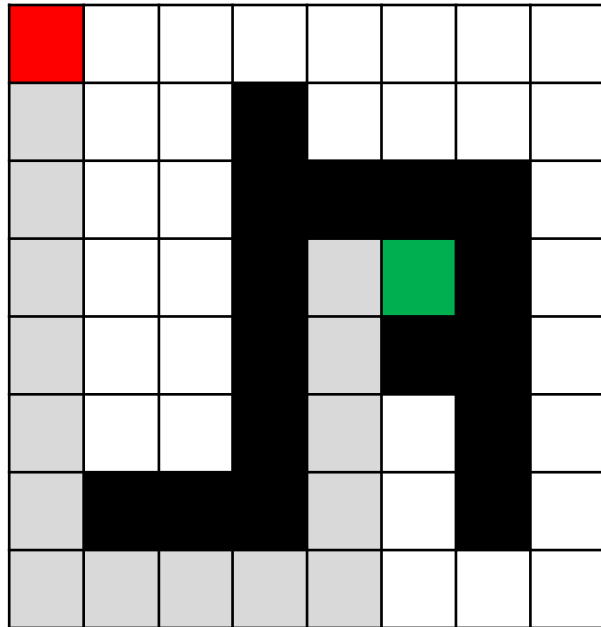
                if neighbor not in cost_so_far or new_cost < cost_so_far[neighbor]:
                    cost_so_far[neighbor] = new_cost # g(n)
                    priority = new_cost + heuristic(neighbor, goal) # f(n) = g(n) + h(n)
                    queue.put((priority, neighbor)) # coloca na fila de prioridades
                    came_from[neighbor] = current # atualiza no anterior de "neighbor"

    return reconstruct_path(came_from, start, goal)

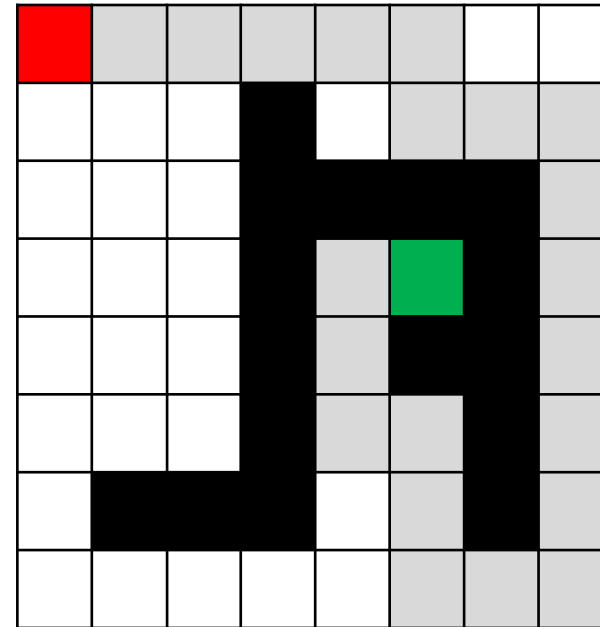
```

Os códigos são muito semelhantes! A única diferença é que, na busca gulosa, não consideramos o custo acumulado, e não temos os processamentos relacionados à lista **cost_so_far**, do algoritmo A*.

Vejamos outro exemplo bastante simples em que o algoritmo A* e a busca gulosa apresentam resultados diferentes:

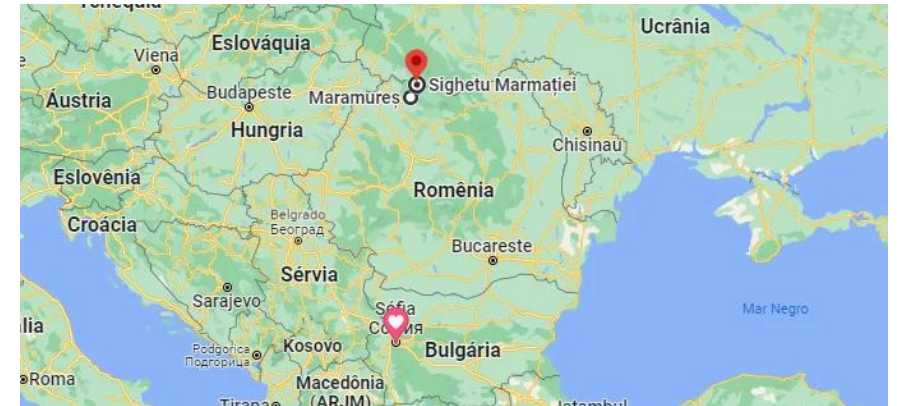
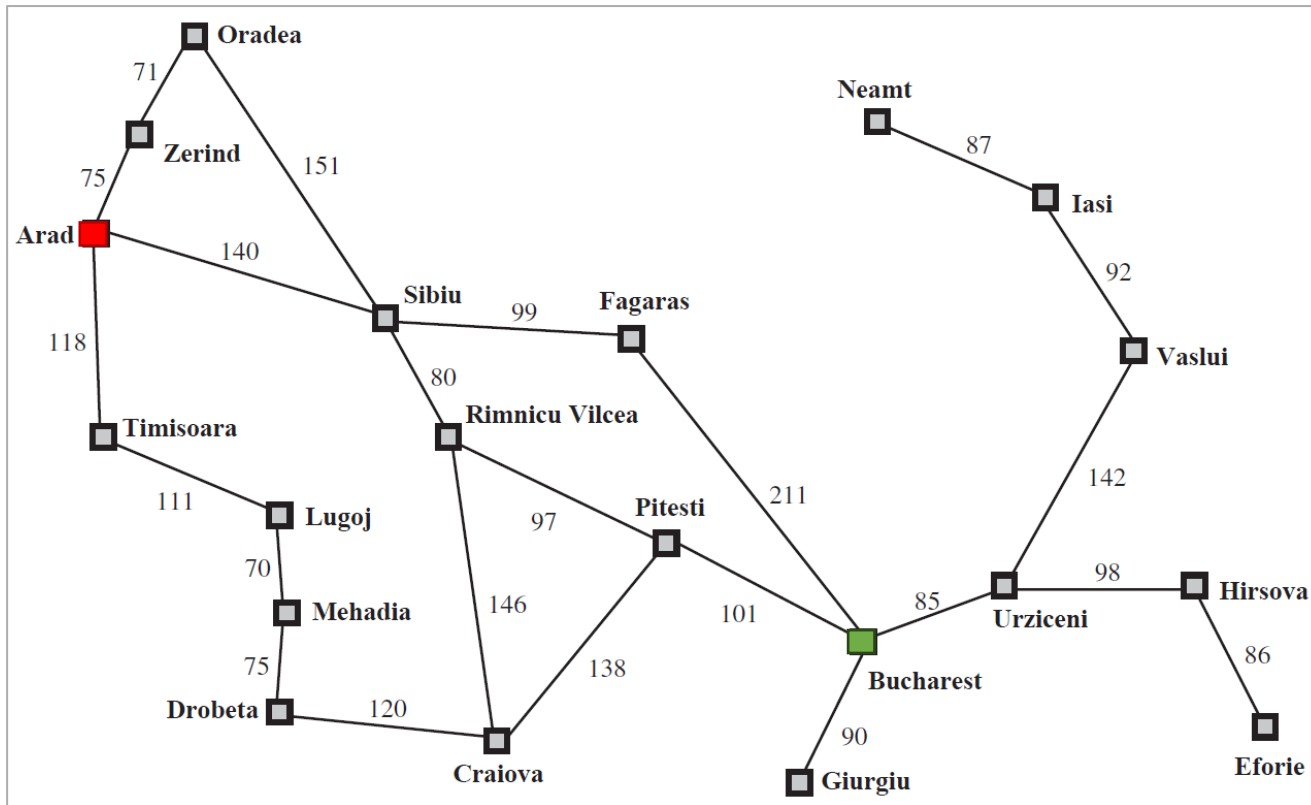


Busca A*: 16 passos.
Solução ótima.



Busca Gulosa: 22 passos.
Solução subótima.

E o problema da Romênia?



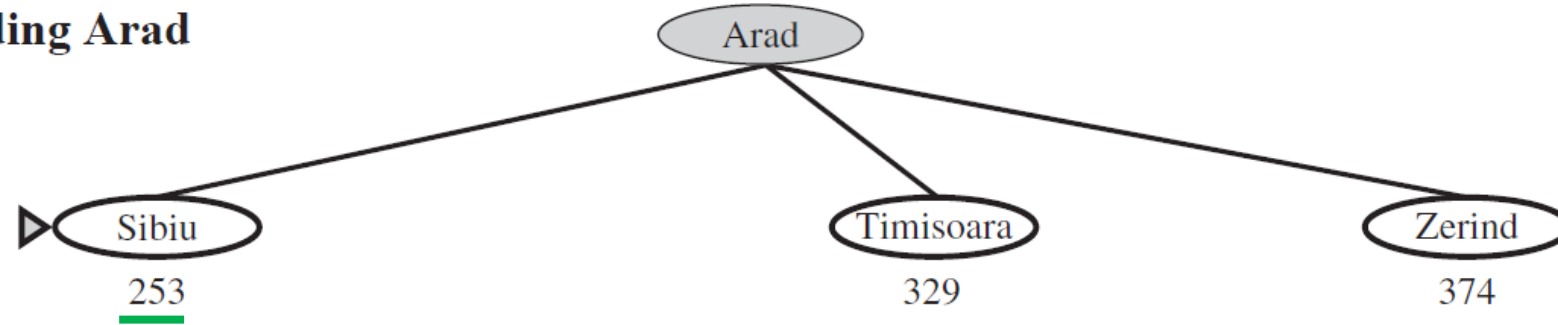
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Novamente, iremos utilizar a **Distância Euclidiana como heurística**, sendo que as distâncias em linha reta entre cada cidade e Bucareste estão na tabela à direita. Agora, com a **Busca Gulosa**.

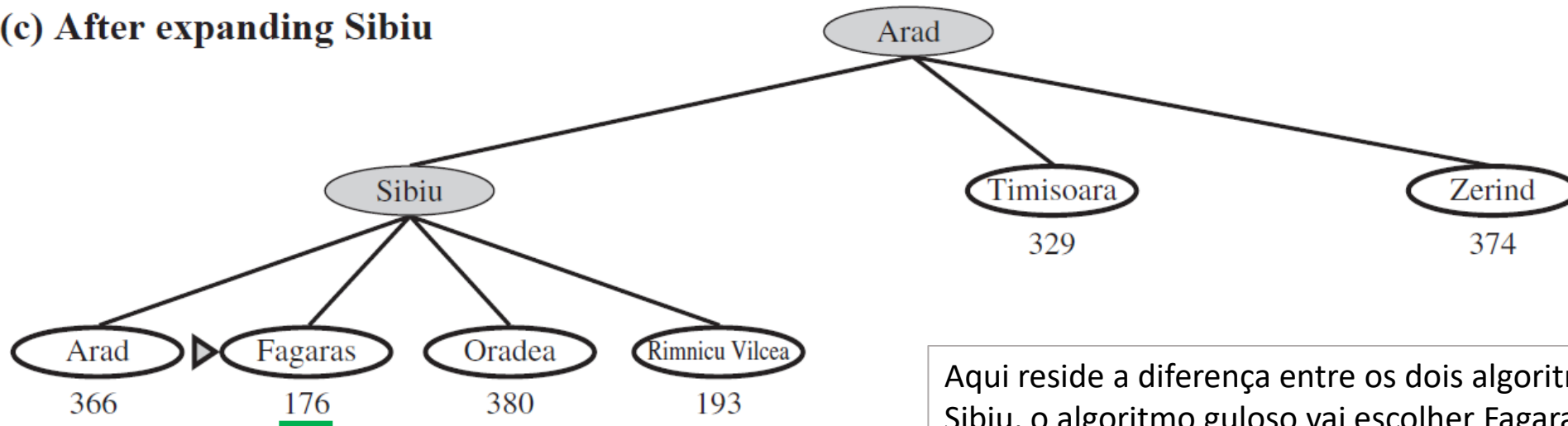
(a) The initial state



(b) After expanding Arad

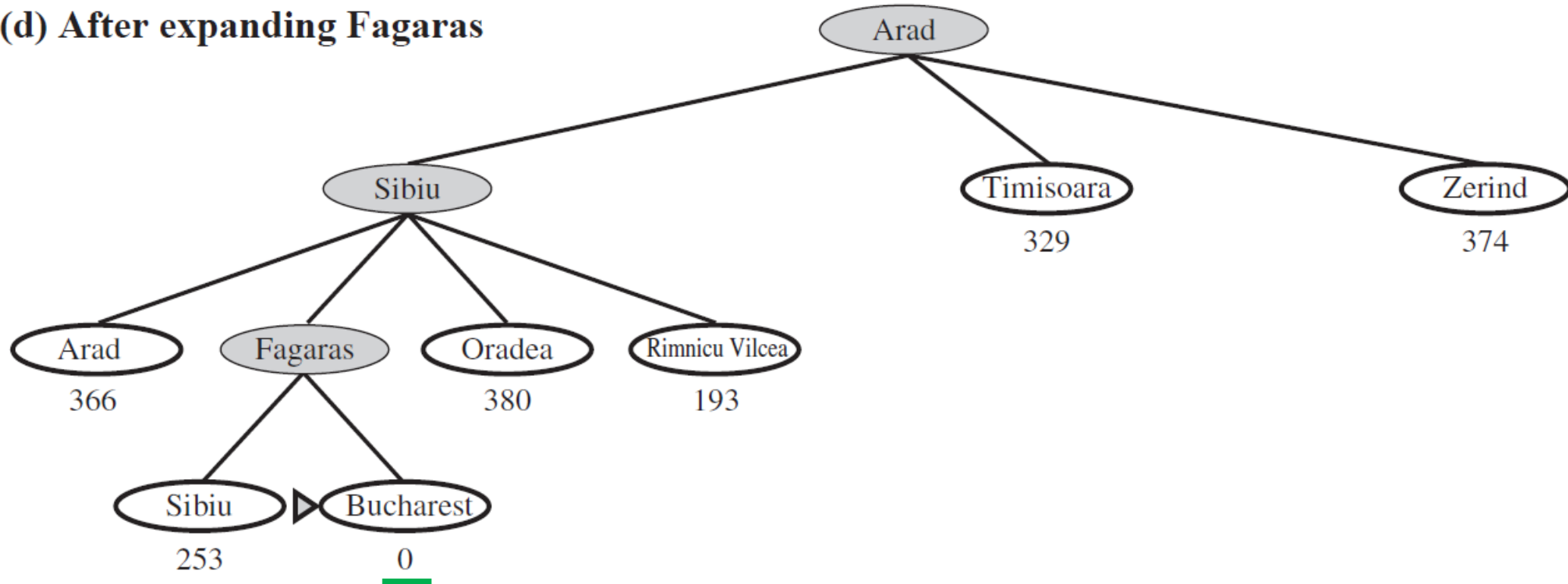


(c) After expanding Sibiu



Aqui reside a diferença entre os dois algoritmos. De Sibiu, o algoritmo guloso vai escolher Fagaras ao invés de Rimnicu. E **nunca mais vai olhar para trás**. Isso gera um **caminho subótimo**.

(d) After expanding Fagaras



Finalmente, o algoritmo chega a Bucareste (com menos passos do que o A*, inclusive), mas utiliza a rota por Fagaras, completando **450 km** percorridos, contra **418 km** do algoritmo A*, **que encontrou o caminho ótimo**. Foi feita uma troca de eficiência (otimização) por velocidade.

```

from queue import PriorityQueue

# Funcao para reconstruir o caminho a partir dos pais
def reconstruct_path(came_from, start, goal):
    current = goal
    path = [current]
    while current != start:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path

# Algoritmo de Busca Gulosa
def greedy_best_first_search(graph, start, goal, heuristic):
    queue = PriorityQueue()
    queue.put((heuristic[start], start))
    came_from = {}
    came_from[start] = None

    while not queue.empty():
        _, current = queue.get()

        if current == goal:
            break

        for neighbor in graph[current]:
            if neighbor not in came_from:
                priority = heuristic[neighbor]
                queue.put((priority, neighbor))
                came_from[neighbor] = current

    return reconstruct_path(came_from, start, goal)

```

```

# Heuristica: Distancia em linha reta ate Bucareste (em km)
heuristic = {
    'Arad': 366, 'Zerind': 374, 'Oradea': 380, 'Sibiu': 253, 'Timisoara': 329,
    'Lugoj': 244, 'Mehadia': 241, 'Drobeta': 242, 'Craiova': 160, 'Rimnicu Vilcea': 193,
    'Fagaras': 176, 'Pitesti': 100, 'Bucharest': 0, 'Giurgiu': 77, 'Urziceni': 80,
    'Hirsova': 151, 'Eforie': 161, 'Vaslui': 199, 'Iasi': 226, 'Neamt': 234
}

# Grafo das cidades romenas e distancias entre elas (em km)
graph = {
    'Arad': {'Zerind': 75, 'Sibiu': 140, 'Timisoara': 118},
    'Zerind': {'Arad': 75, 'Oradea': 71},
    'Oradea': {'Zerind': 71, 'Sibiu': 151},
    'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
    'Timisoara': {'Arad': 118, 'Lugoj': 111},
    'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
    'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
    'Drobeta': {'Mehadia': 75, 'Craiova': 120},
    'Craiova': {'Drobeta': 120, 'Rimnicu Vilcea': 146, 'Pitesti': 138},
    'Rimnicu Vilcea': {'Sibiu': 80, 'Craiova': 146, 'Pitesti': 97},
    'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
    'Pitesti': {'Rimnicu Vilcea': 97, 'Craiova': 138, 'Bucharest': 101},
    'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90, 'Urziceni': 85},
    'Giurgiu': {'Bucharest': 90},
    'Urziceni': {'Bucharest': 85, 'Hirsova': 98, 'Vaslui': 142},
    'Hirsova': {'Urziceni': 98, 'Eforie': 86},
    'Eforie': {'Hirsova': 86},
    'Vaslui': {'Urziceni': 142, 'Iasi': 92},
    'Iasi': {'Vaslui': 92, 'Neamt': 87},
    'Neamt': {'Iasi': 87}
}

# Executa a Busca Gulosa para encontrar o caminho de Arad a Bucareste
start_city = 'Arad'
goal_city = 'Bucharest'
path = greedy_best_first_search(graph, start_city, goal_city, heuristic)

print("Caminho encontrado pela Busca Gulosa:", " -> ".join(path))

```

A Busca Gulosa é um algoritmo simples e rápido que pode ser útil em situações onde uma solução "boa o suficiente" é aceitável, ou onde a **heurística é muito bem alinhada com o espaço de busca**, permitindo que o **algoritmo chegue ao objetivo rapidamente**. No entanto, devido à sua **natureza ambiciosa**, não é ideal para problemas complexos onde é importante garantir a solução ótima.

A comparação com o A* destaca as limitações da Busca Gulosa, especialmente em cenários onde a solução ótima é crítica e onde o espaço de busca tem muitos mínimos locais ou caminhos subótimos. Em tais casos, o A* é preferível devido à sua capacidade de considerar tanto o custo acumulado quanto a estimativa heurística, proporcionando uma busca mais equilibrada e confiável.